Assignment 2: Operating-System Operations and System Protection

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Concepts That Click

Deadline: **two weeks from the date of issue** — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only. Marks are allocated as follows:

Section	Questions	Marks each	Total
Conceptual	Q1–Q2	15	30
Numerical	Q3–Q4	15	30
Design	Q5–Q6	20	40
			100

2.1 Conceptual

Q1. A junior engineer proposes moving the mode bit out of the CPU and into a reserved page of physical memory, arguing that the kernel could then save and restore it more cheaply. The page would be marked "kernel only", and user programs would be told never to write there.

(a) Explain why this design breaks dual-mode operation even if every user program is well behaved on the day it ships. Describe the exact attack that becomes possible.

(b) Classify each operation below as *privileged* or *unprivileged*, and state the single criterion you used:

(i) read the wall clock;
(ii) load the timer with a new value;
(iii) execute the trap instruction;
(iv) clear the interrupt-enable flag;
(v) store to an address inside the process's own address space;
(vi) write the page-table base register;
(vii) add two registers;
(viii) halt the CPU.

(c) What does the hardware do when user code executes a privileged instruction: silently ignore it, or something else? Explain why that behaviour is preferable to a silent no-op.

Q2. The administrator of a multiuser server announces: "Our protection is perfect — the CPU has a mode bit, the MMU checks every access, and all I/O instructions are privileged. Therefore the server is secure."

(a) Distinguish *protection* from *security* as the lecture does. State which one is a *mechanism* and which one is a *policy*, and give one example of each that the other does not cover.

(b) For each incident below, say whether it is a failure of *protection*, a failure of *security policy*, or *neither* (the mechanism working as intended). Justify each classification in one sentence.

(i) A buggy kernel driver writes past its buffer into another process's page.

(ii) Every user account is in the `sudo` group, and one account's password is guessed.

(iii) A user program executes a raw disk I/O instruction.

(iv) An attacker exploits an unpatched network service, obtains a shell, and then reads another user's files through ordinary `read` system calls.

(c) Explain the sentence "A perfect mode bit does not help if the administrator grants every user root access." Then explain why I/O protection is still necessary even though the file system already enforces file permissions.

2.2 Numerical

Q3. A time-sharing kernel arms the timer for a quantum Q and charges $C = 24$ μs per preemption: save the context, run the scheduler, restore a context. Use the lecture's convention, in which the overhead is C/Q and the number of switches per second is $1/Q$. The CPU is otherwise fully busy with user work.

(a) For $Q = 8$ ms, compute the number of switches per second, the switching time consumed per second, and the fraction of CPU time spent switching (as a percentage).

(b) Repeat part (a) for $Q = 800$ μs.

(c) Find the smallest quantum for which switching overhead is at most 1%.

(d) A new CPU costs $C = 60$ μs per switch because it flushes the cache and TLB. Recompute the smallest quantum for 1% overhead, and compute the overhead at the quantum you found in part (c).

(e) In one sentence, connect your answers to the lecture's claim that a tiny quantum "wastes CPU time on context switches" and that real systems choose a quantum in the range 1–100 ms.

Q4. A kernel programs the timer to interrupt f times per second. Each timer interrupt costs $C_t = 30$ μs of CPU time. In a 6 s measurement window the workload also performs $N = 120{,}000$ system calls, and each system call costs $C_s = 5$ μs of kernel time (mode switch, dispatch, argument validation, return). Ignore all other kernel work.

(a) What quantum Q (in milliseconds) does $f = 400$ Hz imply?

Concepts That Click

(b) What fraction of CPU time goes to timer handling at $f = 400$ Hz?

(c) Compute the total kernel time over the 6 s window, including both timer handling and system calls, and express it as a percentage of the window.

(d) The designer doubles f to 800 Hz to improve responsiveness. Recompute parts (b) and (c).

(e) The system-call path is then optimized so that C_s falls from 5 μs to 2 μs. At $f = 800$ Hz, recompute the kernel fraction, and state which term now dominates.

(f) In one sentence, explain why the kernel cannot raise f arbitrarily high to improve response time.

2.3 Design

Q5. A kernel is to gain the system call

```
int map_region(void *addr, size_t len,
    int prot, int fd, off_t offset)
```

It asks the kernel to map `len` bytes of the open file `fd`, starting at byte `offset`, into the calling process's address space at the hint `addr`, with permissions `prot` (read, write, and execute bits). It returns 0 on success and -1 on failure.

(a) Give the complete transition sequence from the user's call to the return, in the style of the lecture's `write` trace. At each step, state whether hardware or kernel performs the work, and mark exactly where the mode bit changes.

(b) List at least five validations the kernel must perform after entry and before it modifies the page tables. For each, state the failure it prevents.

(c) The kernel is already in kernel mode, so it *can* read any physical address. Why is it still wrong to copy from `addr` before validating it? Your answer must refer to what `addr` actually names and to the process's legal address range.

(d) The lecture calls the trap "the only legal door" into the kernel. Explain why the kernel cannot instead be called as a normal function from user code, in terms of the mode bit and the return path.

Q6. A microcontroller in an industrial controller has no programmable interval timer. It does have a watchdog: a hardware counter that resets the board unless the kernel writes a magic value every $W = 50$ ms. The magic write is privileged; the reset is not. The board must run a trusted control loop and an untrusted telemetry process, and the kernel must be able to preempt either of them.

(a) Can the watchdog, used only as a reset device, provide preemption? Design a scheme that does, or explain why none is possible. Your answer must identify the event that returns control to the kernel and state whether user code can mask it.

(b) Suppose the kernel reloads the watchdog only at the end of each system call. The telemetry process then enters `while(1);` and makes no further system call. Trace what happens at $t = 50$ ms, at $t = 100$ ms, and after the reset. Does this provide preemption? Justify your answer.

Concepts That Click

(c) A colleague proposes making the magic-value write unprivileged so that the telemetry process can "help" keep the board alive. Explain the attack this enables and why it defeats the purpose of the watchdog.

(d) Recommend a fix: name the hardware feature you would add, and state the two properties it must have for the lecture's definition of preemption to be satisfied. Justify each property in one sentence.

Reading. The lecture follows Silberschatz et al. [1], Ch. 1 §1.5, pp. 20–23 (dual-mode operation, timer, protection), and Stallings [2], Ch. 3 §3.5–3.6, pp. 140–143 (execution of the OS, security issues).

References

[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[2] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click